INDEX

S.No	Title	Page number
1	INTRODUCTION	4
2	OVERVIEW	5
3	COMPONENTS REQUIRED	6
4	BLOCK DIAGRAM	7
5	EXPLANATION	8
6	PROGRAM	9
7	OUTPUT	14
8	VIDEO LINK & GEO TAG	16
9	CONCLUSION	17
10	REFERENCES	18

INTRODUCTION:

The Smart Occupancy Detection System using MSP430 and ESP32 is an Embedded Systems and Internet of Things (IoT) based project designed to automatically detect and monitor the occupancy status of a particular room or area. In conventional environments, determining whether a room is occupied or vacant often depends on manual observation, which can result in inefficient utilization of spaces and unnecessary energy consumption. The proposed system provides an automated method for detecting human presence and transmitting occupancy information through a wireless network. It can be used in classrooms, offices, laboratories, homes, meeting rooms, and other indoor environments.

The core of the system is the Texas Instruments MSP430 microcontroller**,** which acts as the main processing and control unit. The MSP430 is a low-power microcontroller suitable for sensing, processing, and control applications. An occupancy or presence sensor detects the presence of a person in the monitored area and provides input to the MSP430. The microcontroller processes this information and determines whether the area is occupied or vacant. Its low-power operation makes the system suitable for continuous monitoring applications.

The system also incorporates the ESP32 Wi-Fi module to provide wireless connectivity and IoT functionality. Once the MSP430 determines the occupancy status, the information is transferred to the ESP32, which connects to a Wi-Fi network and transmits the data to a remote monitoring platform. This allows occupancy information to be accessed through a smartphone, computer, or other connected device. The integration of the MSP430 and ESP32 combines low-power embedded processing with wireless IoT communication.

The Smart Occupancy Detection System can also support energy management and smart building applications. Occupancy information can be used to control electrical appliances such as lights, fans, and air-conditioning systems according to whether a room is occupied or vacant. When no person is detected, the system can help reduce unnecessary energy usage. Occupancy data can also be useful for monitoring the utilization of classrooms, offices, laboratories, and other facilities.

This project demonstrates the integration of sensors, microcontrollers, wireless communication, and IoT technology into a single monitoring system. The combination of the MSP430 and ESP32 provides a practical method for detecting occupancy locally and communicating the information remotely. The system can be further extended with IoT dashboards, cloud-based data storage, multiple sensors, alerts, and automatic appliance control. Thus, the proposed system provides a practical solution for real-time occupancy monitoring in smart environments.

OVERVIEW:

The Smart Occupancy Detection System using MSP430 and ESP32 is an embedded systems and IoT project designed to automatically detect whether a particular room or area is occupied. The system reduces the need for manual monitoring by using a sensor to identify the presence of people and providing the occupancy status through a simple and automated process. It can be used in classrooms, offices, laboratories, homes, and other indoor environments.

The system is based on the MSP430 microcontroller**,** which acts as the central processing unit. It receives the signal from the occupancy or presence sensor and processes the information to determine whether a person is present. Based on the sensor input, the system identifies the area as either occupied or vacant. The low-power operation of the MSP430 makes it suitable for continuous monitoring applications.

The ESP32 Wi-Fi module is used to provide wireless communication and IoT connectivity. The occupancy status processed by the MSP430 is sent to the ESP32, which connects to a Wi-Fi network and transmits the information to a remote monitoring platform. This allows the user to monitor the occupancy status using a connected device such as a smartphone or computer.

The working of the system involves continuously monitoring the selected area through the occupancy sensor. When a person is detected, the MSP430 processes the sensor signal and generates an occupied status. When no person is detected, the system identifies the area as vacant**.** The corresponding information is then communicated through the ESP32, allowing the occupancy condition to be monitored remotely.

Overall, the Smart Occupancy Detection System provides a simple and practical application of embedded systems and IoT technology. It can help in monitoring space utilization and can be further expanded with features such as automatic light and fan control, cloud-based data storage, occupancy history, multiple sensors, and real-time notifications. This makes the system a useful foundation for developing smart classrooms, smart offices, and energy-efficient smart buildings.

COMPONENTS REQUIRED:

• MSP430 LaunchPad – Main development board with built-in MSP430 microcontroller, used to process sensor input and control the entire system.

• ESP32 ESP-01 – Wi-Fi communication module used to transmit occupancy information to an IoT platform.

• PIR Motion Sensor HC-SR501 – Used to detect human presence or movement in the monitored area.

• LED – Provides visual indication when the area is occupied.

• 220 Ω Resistor – Used to limit current and protect the LED from excessive current.

• Buzzer – Provides an optional audible alert based on the occupancy condition.

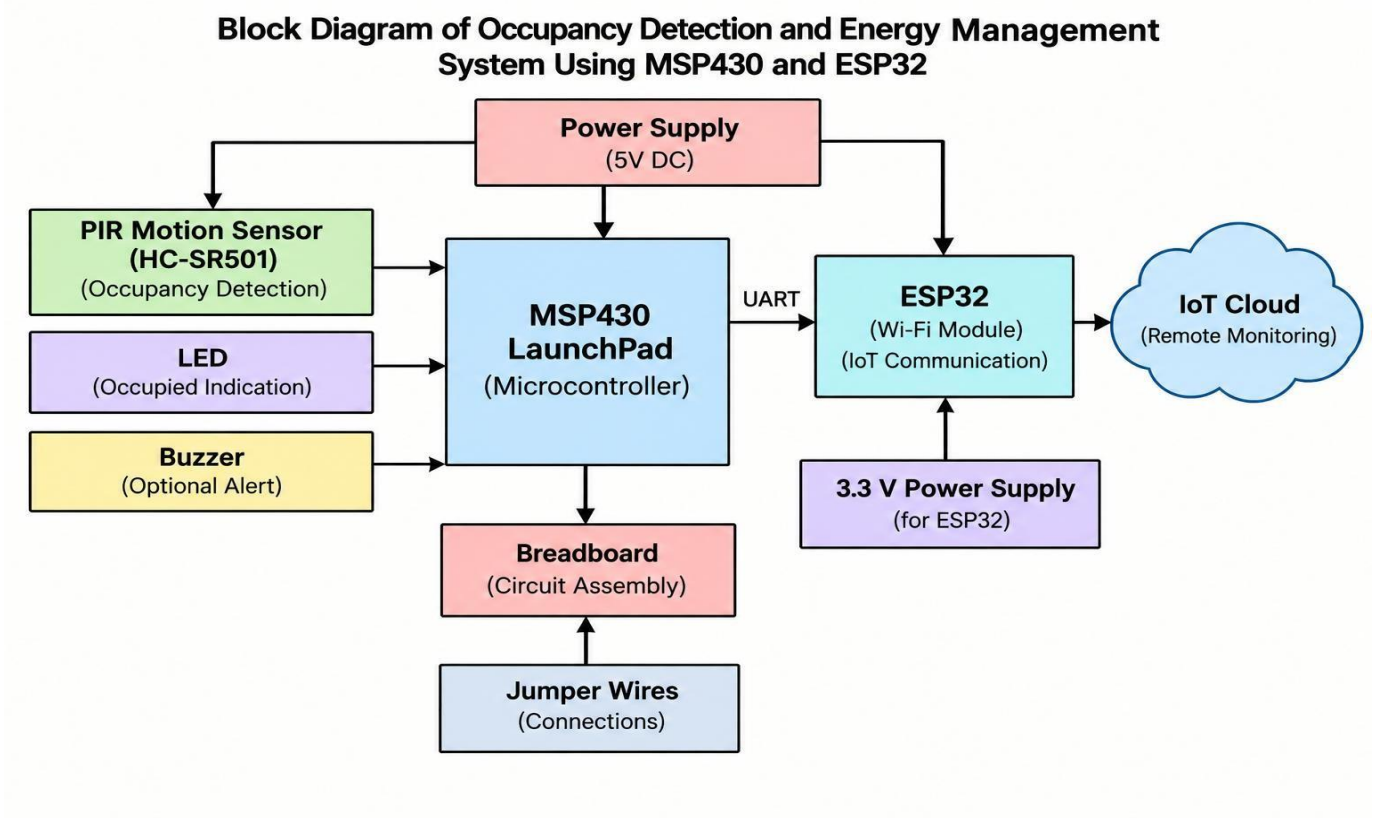
• Breadboard – Used for assembling and testing the circuit without soldering.

• Jumper Wires – Used to establish connections between the MSP430, sensor, ESP32, LED, and other components.

• USB Cable – Used for programming and supplying power to the MSP430 LaunchPad.

• 3.3 V Power Supply – Provides the required 3.3 V supply for the ESP32 Wi-Fi module.

BLOCKDIAGRAM:



EXPLANATION:

1. **Occupancy Detection (Presence Detection)**
   - The system uses a PIR motion sensor to detect whether a person is present in the monitored area.
   - This input represents the occupied/vacant condition of the room.
   - If a person is detected, the system identifies the area as occupied.
   - If no person is detected, the system considers the area vacant.
2. **Signal Processing via MSP430 Microcontroller**
   - The MSP430 microcontroller continuously reads the input from the PIR sensor.
   - If occupancy is detected, the system processes the signal as an occupied condition.
   - If occupancy is not detected, the system processes the signal as a vacant condition.
   - The MSP430 controls the status indication and further communication of the system.
3. **Occupancy Status Indication**
   - Based on the sensor input, the microcontroller provides the current occupancy status.
   - An **LED** is used to provide a visual indication when the area is occupied.
   - The system can indicate conditions such as **"**ROOM OCCUPIED" or "ROOM VACANT."
   - This provides immediate information about the monitored area.
4. **Wireless Communication using ESP32**
   - The occupancy status processed by the MSP430 is transferred to the ESP32 ESP-01 module.
   - The ESP32 connects to an available Wi-Fi network for wireless communication.
   - It transmits the occupancy information to the selected IoT monitoring platform.
   - This allows the occupancy status to be monitored remotely.
5. **Real-Time IoT Monitoring**
   - The ESP32 sends the updated occupancy information through the Internet.
   - The user can view the occupancy status using a smartphone, computer, or IoT dashboard.
   - Whenever the occupancy condition changes, the updated status is transmitted.
   - This provides real-time monitoring of the selected area.
6. **Energy Management**
   - The occupancy information can be used for automatic control of electrical appliances.
   - When the area is occupied, appliances such as lights or fans can be switched ON.
   - When the area becomes vacant, the appliances can be switched OFF after a specified duration.
   - This helps reduce unnecessary energy consumption.
7. **Continuous Operation Loop**
   - The microcontroller continuously monitors the PIR sensor while the system is powered.
   - It repeatedly processes the sensor input and updates the occupancy status.
   - The ESP32 transmits the updated information through the Wi-Fi network.
   - This ensures that the system remains active and provides continuous occupancy monitoring.

PROGRAM:

```
 #include <WiFi.h>
#include <HTTPClient.h>

// ========================================
// Wi-Fi Details
// ========================================

const char* ssid = "Capstone";
const char* password = "12345678";

// ========================================
// ThingSpeak Write API Key
// ========================================

const char* thingSpeakAPI = "LEXCO3WV2T1OSJU0";

// ========================================
// MSP430 UART
// ========================================

#define RXD2 16
#define TXD2 17

HardwareSerial MSPSerial(2);

// ========================================
// Variables
// ========================================

String occupancy = "VACANT";

unsigned long previousMillis = 0;

// ThingSpeak minimum interval
const unsigned long uploadInterval = 1000;


// ========================================
// SETUP
// ========================================

void setup()
```

```
{
  Serial.begin(115200);

  MSPSerial.begin(9600, SERIAL_8N1, RXD2, TXD2);

  Serial.println();
  Serial.println("====================================");
  Serial.println(" SMART OCCUPANCY DETECTION SYSTEM");
  Serial.println(" MSP430 + ESP32 + THINGSPEAK");
  Serial.println("====================================");

  // Connect to Wi-Fi

  WiFi.begin(ssid, password);

  Serial.print("Connecting to Wi-Fi");

  while (WiFi.status() != WL_CONNECTED)
  {
    delay(500);
    Serial.print(".");
  }

  Serial.println();
  Serial.println("Wi-Fi Connected!");

  Serial.print("IP Address: ");
  Serial.println(WiFi.localIP());

  Serial.println();
  Serial.println("Waiting for MSP430 data...");
}


// ========================================
// LOOP
// ========================================

void loop()
{
  // Read data from MSP430

  if (MSPSerial.available())
  {
    String data = MSPSerial.readStringUntil('\n');
```

```
    data.trim();

    Serial.print("MSP430 DATA: ");
    Serial.println(data);

    if (data == "OCCUPIED")
    {
      occupancy = "OCCUPIED";
    }
    else if (data == "VACANT")
    {
      occupancy = "VACANT";
    }
  }


  // Upload every 15 seconds

  if (millis() - previousMillis >= uploadInterval)
  {
    previousMillis = millis();

    sendToThingSpeak();
  }
}


// ========================================
// SEND DATA TO THINGSPEAK
// ========================================

void sendToThingSpeak()
{
  if (WiFi.status() != WL_CONNECTED)
  {
    Serial.println("Wi-Fi disconnected!");

    WiFi.reconnect();

    return;
  }

  int occupancyValue;

  if (occupancy == "OCCUPIED")
  {
```

```
  occupancyValue = 1;
}
else
{
  occupancyValue = 0;
}


Serial.println();
Serial.println("Uploading to ThingSpeak...");


String url = "http://api.thingspeak.com/update?api_key=";

url += thingSpeakAPI;

url += "&field1=";

url += String(occupancyValue);


HTTPClient http;

http.begin(url);

int httpCode = http.GET();


if (httpCode > 0)
{
  Serial.print("HTTP Response: ");
  Serial.println(httpCode);

  String response = http.getString();

  Serial.print("ThingSpeak Response: ");
  Serial.println(response);


  if (response != "0")
  {
    Serial.println("Upload SUCCESS");
  }
  else
  {
    Serial.println("Upload FAILED");
```

```
    }
  }
  else
  {
    Serial.print("HTTP Error: ");
    Serial.println(httpCode);
  }


  http.end();


  Serial.print("Occupancy: ");
  Serial.println(occupancy);

  Serial.print("Field 1 Value: ");
  Serial.println(occupancyValue);

  Serial.println("----------------------");
}
```

OUTPUT:

_04_05_06_capstine_ino

```
  Serial Monitor:
  Baud Rate = 9600
*/

#define PIR_PIN P1_3
#define LED_PIN P1_0

void setup()
{
  pinMode(PIR_PIN, INPUT);
  pinMode(LED_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);

  Serial.begin(9600);

  delay(1000);

  Serial.println("SMART OCCUPANCY DETECTION SYSTEM");
  Serial.println("---------------------------------");
  Serial.println("System Started");
  Serial.println("Waiting for PIR sensor...");
  Serial.println();
}

void loop()
{
  int pirState = digitalRead(PIR_PIN);
```

COM9

```
VACAVACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
OCCUPIED - Person Detected
OCCUPIED - Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
VACANT - No Person Detected
```

Autoscroll

```
Done Saving.
The sketch name had to be modified. Sketch names can only consist
of ASCII characters and numbers (but cannot start with a number).
They should also be less less than 64 characters long.

29
```

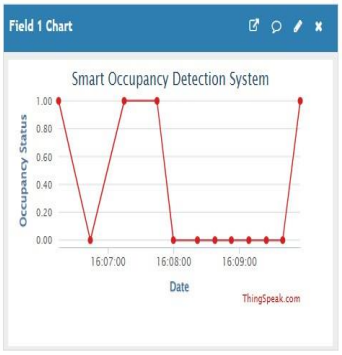
ThingSpeak™ Channels Apps Devices Support Commercial Use How to Buy AS

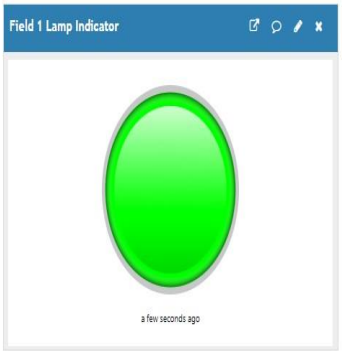
Channel 3 of 3

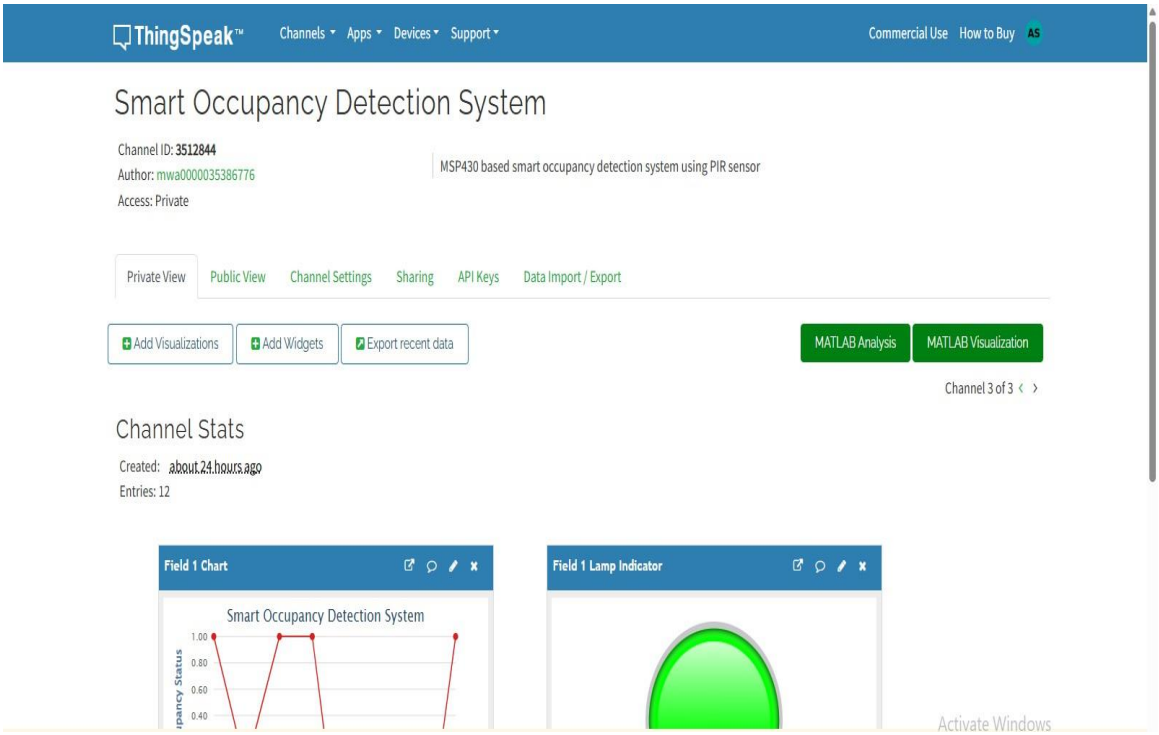
Channel Stats

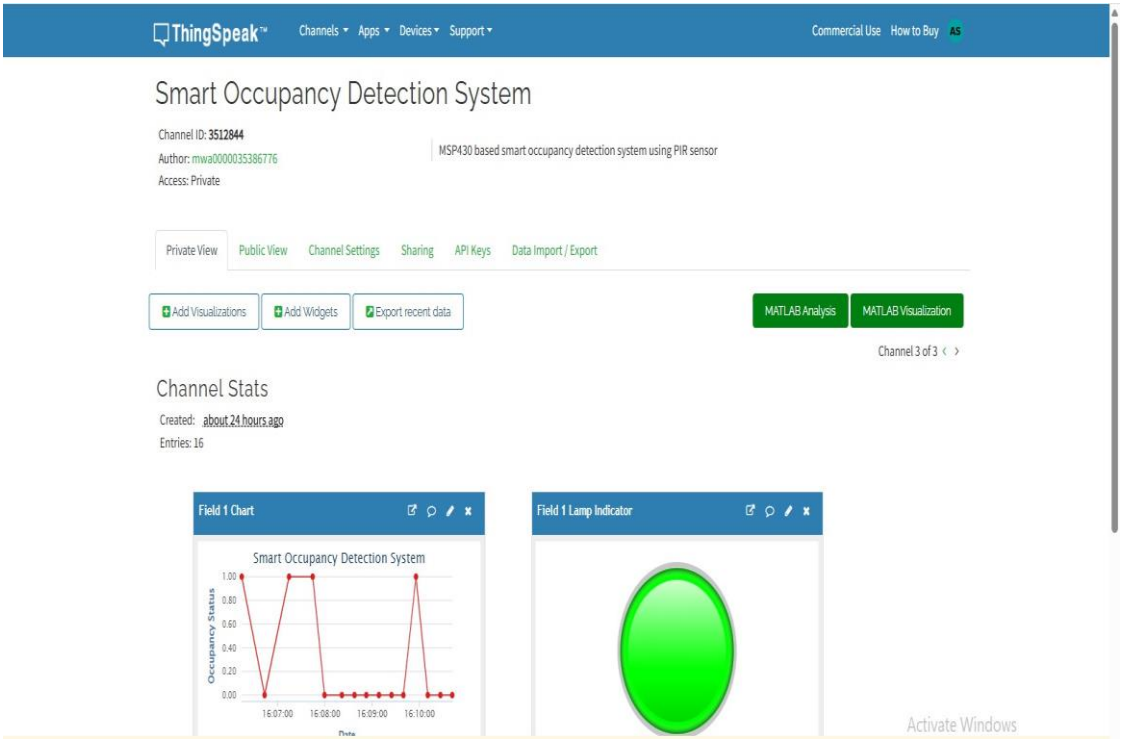
Created: about 24 hours ago

Entries: 12



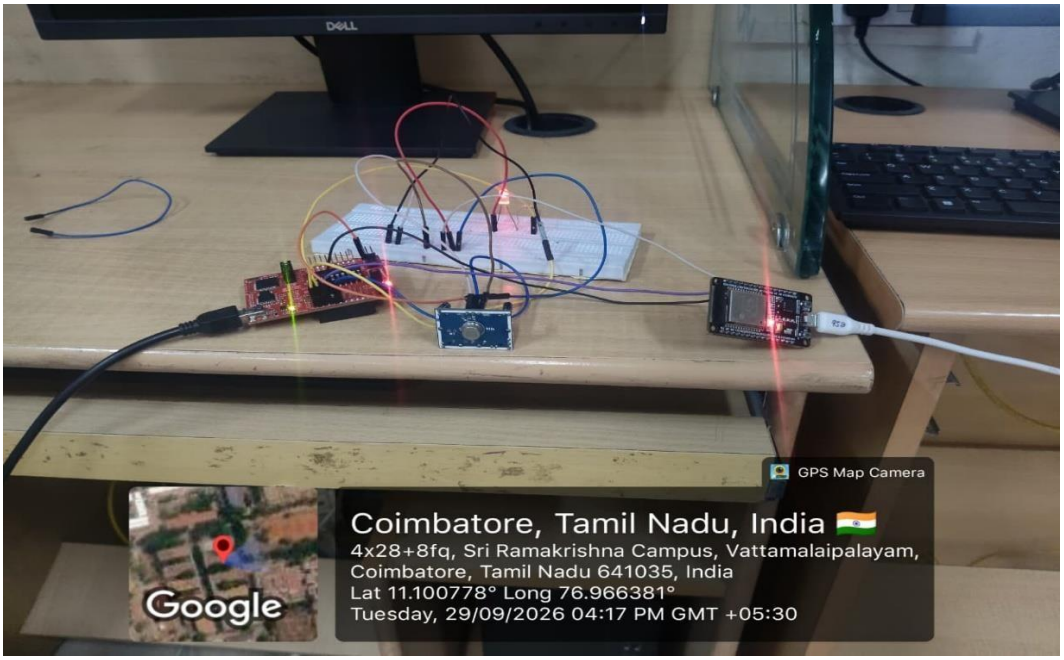


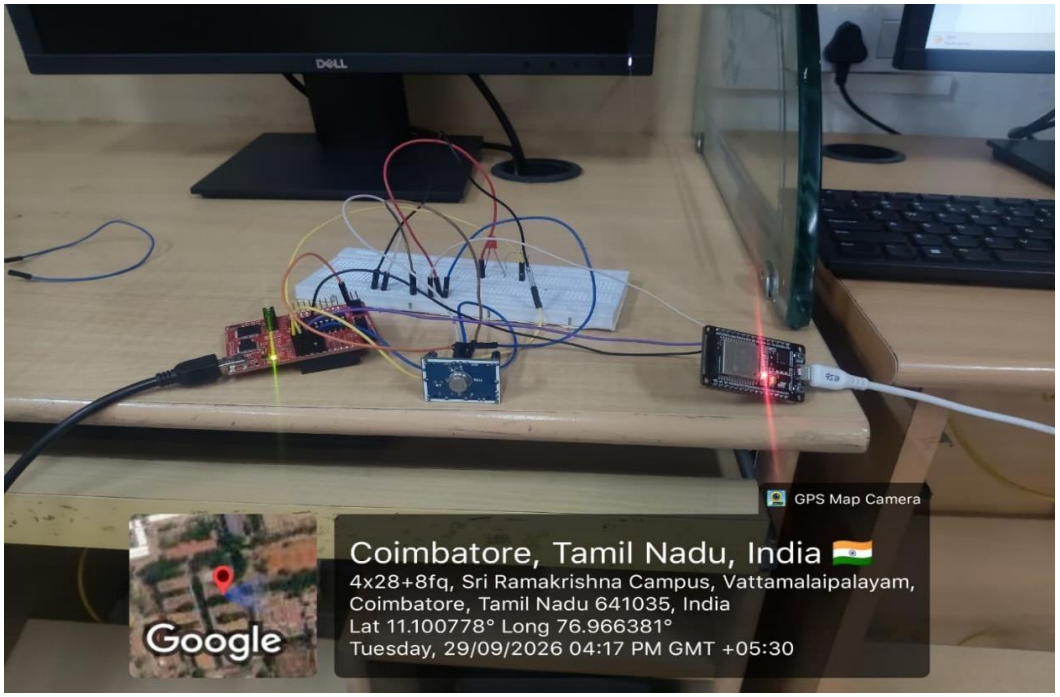

ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Smart Occupancy Detection System
Channel ID: 3512844
Author: mwa0000035386776
Access: Private
MSP430 based smart occupancy detection system using PIR sensor
Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export
Add Visualizations
Add Widgets
Export recent data
MATLAB Analysis
MATLAB Visualization
Channel 3 of 3
Channel Stats
Created: about 24 hours ago
Entries: 12
Field 1 Chart
Smart Occupancy Detection System
Occupancy Status
1.00
0.80
0.60
0.40
Field 1 Lamp Indicator
Activate Windows


ThingSpeak™
Channels
Apps
Devices
Support
Commercial Use
How to Buy
Smart Occupancy Detection System
Channel ID: 3512844
Author: mwa0000035386776
Access: Private
MSP430 based smart occupancy detection system using PIR sensor
Private View
Public View
Channel Settings
Sharing
API Keys
Data Import / Export
Add Visualizations
Add Widgets
Export recent data
MATLAB Analysis
MATLAB Visualization
Channel 3 of 3
Channel Stats
Created: about 24 hours ago
Entries: 16
Field 1 Chart
Smart Occupancy Detection System
Occupancy Status
1.00
0.80
0.60
0.40
0.20
0.00
16:07:00
16:08:00
16:09:00
16:10:00
Field 1 Lamp Indicator
Activate Windows

VIDEO LINK:

https://youtu.be/p-WfVClxeYo?si=hJnl0-R32zlP7ReJ





CONCLUSION:

The Smart Occupancy Detection System using MSP430 and ESP32 provides a simple and effective solution for automatically monitoring the occupancy status of a room or designated area. The system demonstrates how embedded systems and IoT technologies can be combined to detect human presence and provide occupancy information through wireless communication.

The implementation using the MSP430 microcontroller enables efficient processing of sensor data while maintaining low power consumption. The ESP32 Wi-Fi module adds IoT connectivity, allowing the detected occupancy information to be transmitted and monitored remotely. This makes the system suitable for applications such as classrooms, offices, laboratories, homes, and smart buildings.

Although the current system mainly focuses on occupancy detection and monitoring, it provides a strong foundation for further improvements. Additional features such as automatic control of lights and fans, cloud-based data storage, real-time notifications, occupancy history, and multiple sensor integration can be incorporated to enhance the functionality of the system.

In conclusion, the project demonstrates the practical application of sensors, MSP430 microcontroller, ESP32 Wi-Fi communication, and IoT technology for real-time occupancy monitoring. It provides a cost-effective and scalable approach that can contribute to better space utilization and energy management in smart environments.

REFERENCES:

1. R. N. Diengdoh et al., "IoT Based Smart Room for Power Consumption Monitoring and Control," *International Journal of Computer Sciences and Engineering*, 2019.
2. Radhika et al., "Smart Room Automation Using IoT," *International Journal of Multidisciplinary Research and Technology*, 2021.
3. G. Mahalakshmi, M. Vigneshwaran, "IoT Based Home Automation Using Arduino," *International Journal of Engineering and Advanced Research Technology*, 2017.
4. "Implementation of Room Automation with Cloud Based Monitoring System," *IEEE Conference Proceedings*, 2018.
5. "An IoT-Based Smart Lighting System for Real-Time Occupancy Monitoring and Energy Management," *European Journal of Advances in Engineering and Technology*, 2022.
6. "Occupancy-Aware Energy Optimization in Educational Infrastructure using IoT," *International Journal of Novel Trends and Innovation*, 2026.
7. Texas Instruments, **"**MSP430 Family User's Guide," Texas Instruments (Technical Reference).
8. Espressif Systems, **"**ESP8266EX Datasheet**,"** Espressif Systems (Technical Reference).
9. "Smart Room Technology Implementation Based on Internet of Things Toward Smart Campus," *Journal of Physics: Conference Series*, 2020.
10. Texas Instruments, **"**MSP430 Microcontroller and Embedded Systems**,"** Texas Instruments (Technical Reference).